

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационная безопасность

Лабораторная работа № 1

Разработка защищенного REST API с интеграцией в CI/CD

Выполнил: Елисеев Константин Иванович

Группа: P3308

Санкт-Петербург
2026

Выполнение

Ссылка на GitHub репозиторий

<https://github.com/virdxk/infosec-lab1>

Описание проекта

Небольшой REST API на Python: вход по логину и паролю с выдачей JWT, чтение и создание постов. Стек: Flask, SQLite через SQLAlchemy, PyJWT, bcrypt.

Таблица 1 - Структура проекта

Файл	Что делает
app/__init__.py	Создание приложения, проверка длины секрета, подключение blueprints
app/config.py	Настройки: секрет и адрес БД из .env, срок токена, стоимость bcrypt
app/auth.py	Вход по логину и паролю, выдача токена
app/security.py	Выпуск и проверка JWT
app/middleware.py	Проверка JWT перед маршрутами /api/*
app/api.py	Чтение и создание постов
app/models.py	Модели User и Post, хэширование паролей bcrypt
app/sanitize.py	Экранирование пользовательских данных в ответах
seed.py	Демо-пользователи alice и bob, два поста

Запуск

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
cp .env.example .env # вписать JWT_SECRET, сгенерировать командой из файла
python seed.py       # пользователи alice и bob, два поста
python wsgi.py        # http://127.0.0.1:5000
```

В .env задаются только JWT_SECRET и DATABASE_URL. Срок жизни токена (3600 с) и стоимость bcrypt (12) зафиксированы в app/config.py.

Описание API

Таблица 2 - Эндпоинты API

Метод и адрес	Назначение	Токен	Успешный ответ
POST /auth/login	Вход, тело {"username", "password"}	не нужен	200 {"access_token"}
GET /api/data	Список постов	нужен	200 [{id, title, body, author}]
POST /api/posts	Создать пост, тело {"title", "body"}	нужен	201 {id, title, body, author}

Токен передается в заголовке Authorization: Bearer <token>. Ошибки: 400 при неполном теле запроса, 401 при неверном пароле или отсутствующем, поддельном либо просроченном токене. В примерах \$BASE_URL - адрес сервера, \$TOKEN - токен из ответа на вход.

POST /auth/login

```
curl -s -X POST "$BASE_URL/auth/login" \
  -H 'Content-Type: application/json' \
  -d '{"username": "alice", "password": "Alice-Str0ng-Pass!"}'

HTTP 200
{
  "access_token": "<JWT>"
}
```

GET /api/data

```
curl -s "$BASE_URL/api/data" -H "Authorization: Bearer $TOKEN"

HTTP 200
[
  {
    "author": "alice",
    "body": "First post.",
    "id": 1,
    "title": "Welcome"
  },
  {
    "author": "bob",
    "body": "Tokens expire in one hour.",
    "id": 2,
    "title": "Notes on JWT"
  }
]
```

POST /api/posts

```
curl -s -X POST "$BASE_URL/api/posts" \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"title": "Hello", "body": "World"}'

HTTP 201
{
  "author": "alice",
  "body": "World",
  "id": 3,
  "title": "Hello"
}
```

Описание реализованных мер защиты

Защита от SQL-инъекций

Запросы к базе строятся через ORM SQLAlchemy. Логин передается в SQL связанным параметром, поэтому ввод ' OR 1=1 -- остается обычной строкой логина. SQL из строк нигде не склеивается.

app/auth.py

```
with current_app.session_factory() as session:
    user = session.scalar(select(User).where(User.username == username))

if user is None or not user.check_password(password):
    return jsonify({"error": "invalid credentials"}), 401
```

Защита от XSS

Перед отдачей клиенту заголовок, текст поста и имя автора экранируются функцией `markupsafe.escape`. В базе хранится исходный текст, экранирование применяется к каждому ответу.

app/sanitize.py

```
def serialize_post(post: Post) -> dict:
    return {
        "id": post.id,
        "title": str(escape(post.title)),
        "body": str(escape(post.body)),
        "author": str(escape(post.author.username)),
    }
```

Проверка

```
curl -s -X POST "$BASE_URL/api/posts" \
-H "Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"title": "XSS", "body": "<script>alert(1)</script>"}'

HTTP 201
{
  "author": "alice",
  "body": "&lt;script&gt;alert(1)&lt;/script&gt;",
  "id": 4,
  "title": "XSS"
}
```

Хранение паролей: bcrypt

В базе хранятся только bcrypt-хэши. Соль генерируется случайно для каждого пароля и хранится внутри хэша, стоимость - 12 раундов. Пароли длиннее 72 байт bcrypt не принимает, поэтому для них проверка сразу возвращает отказ.

app/models.py

```
def set_password(self, password: str, rounds: int) -> None:
    salt = bcrypt.gensalt(rounds=rounds)
    self.password_hash = bcrypt.hashpw(password.encode(), salt).decode()

def check_password(self, password: str) -> bool:
    password_bytes = password.encode()
    # bcrypt raises ValueError on passwords longer than 72 bytes
    if len(password_bytes) > 72:
        return False
    return bcrypt.checkpw(password_bytes, self.password_hash.encode())
```

На неверный пароль и на несуществующего пользователя ответ одинаковый: `401 {"error": "invalid credentials"}`.

Аутентификация: JWT

После входа выдается токен HS256 с полями `sub` (ID пользователя), `iat` и `exp` (срок 1 час). Секрет берется из `.env`; если он короче 32 символов, приложение не запускается. При проверке алгоритм задан сервером, поэтому токен с `alg: none` отклоняется.

app/security.py

```
def issue_token(user_id: int) -> str:
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(user_id),
        "iat": now,
        "exp": now + timedelta(seconds=current_app.config["JWT_TTL_SECONDS"]),
    }
    return jwt.encode(payload, current_app.config["JWT_SECRET"], algorithm=ALGORITHM)

def decode_token(token: str) -> dict:
    return jwt.decode(
        token,
        current_app.config["JWT_SECRET"],
        algorithms=[ALGORITHM],
        options={"require": ["sub", "iat", "exp"]},
    )
```

Middleware подключено ко всем маршрутам `/api/*` через хук `before_request`. Оно проверяет заголовок, подпись и срок токена, находит пользователя в базе и только тогда пропускает запрос, иначе отвечает 401. Автор нового поста берется из токена, а не из тела запроса.

app/middleware.py

```
def authenticate_request():
    header = request.headers.get("Authorization", "")
    if not header.startswith("Bearer "):
        return jsonify({"error": "missing token"}), 401

    try:
        claims = decode_token(header.removeprefix("Bearer "))
    except jwt.InvalidTokenError:
        return jsonify({"error": "invalid token"}), 401

    with current_app.session_factory() as session:
        user = session.get(User, int(claims["sub"]))
    if user is None:
        return jsonify({"error": "invalid token"}), 401

    g.user_id = user.id
    return None
```

Проверка API через curl

Сервер запущен локально на чистой базе после `python seed.py`. Полный протокол с командами и ответами - файл `docs/curl-session.md` в репозитории.

Таблица 3 - Результаты проверки через curl

№	Сценарий	Код	Ответ
1	Вход alice с верным паролем	200	access_token
2	Неверный пароль	401	invalid credentials
3	SQL-инъекция в логине	401	invalid credentials
4	GET /api/data без токена	401	missing token
5	Токен без подписи (alg: none)	401	invalid token
6	GET /api/data с токеном	200	список из двух постов
7	Создание поста	201	пост с id 3
8	Пост со <script>	201	body экранирован
9	Пост без поля body	400	title and body are required

Сценарий 3: SQL-инъекция

```
curl -s -X POST "$BASE_URL/auth/login" \
  -H 'Content-Type: application/json' \
  -d '{"username": "'" OR 1=1 --\',"password": "x"}'
```

```
HTTP 401
{
  "error": "invalid credentials"
}
```

CI/CD pipeline

Файл `.github/workflows/ci.yml` запускается на каждый push и pull request. В нем три независимые задачи, любая найденная проблема завершает pipeline ошибкой.

Таблица 4 - Задачи pipeline

Задача	Что делает	Когда падает
Tests	<code>pytest -v</code> : вход, SQL-инъекция, хэш пароля, доступ без токена, поддельный, <code>alg: none</code> и просроченный токены, создание поста, XSS	Упал хотя бы один тест
SAST (bandit)	Bandit по <code>app</code> , <code>seed.py</code> , <code>wsgi.py</code> , текстовый отчет в артефакт <code>bandit-report</code>	Замечание уровня <code>medium</code> или <code>high</code>
SCA (Dependency-Check)	OWASP Dependency-Check 13.0.0 по всем зависимостям, включая транзитивные, HTML-отчет в артефакт <code>dependency-check-report</code>	Уязвимость с CVSS 7 и выше

Результаты запуска CI #12 (коммит 1fd3837):

- Tests: **14 тестов пройдено**;
- Bandit: **0 замечаний** (175 строк кода);
- OWASP Dependency-Check: **13 зависимостей, 0 уязвимостей**, одно ложное срабатывание исключено.

Исключенное срабатывание - CVE-2025-45770. Оно относится к PHP-библиотеке lcobucci/jwt, а сканер сопоставил его с Python-пакетом PyJWT. Исключение описано в .github/dependency-check-suppressions.xml и ограничено этим пакетом и CVE.

Отчеты из pipeline

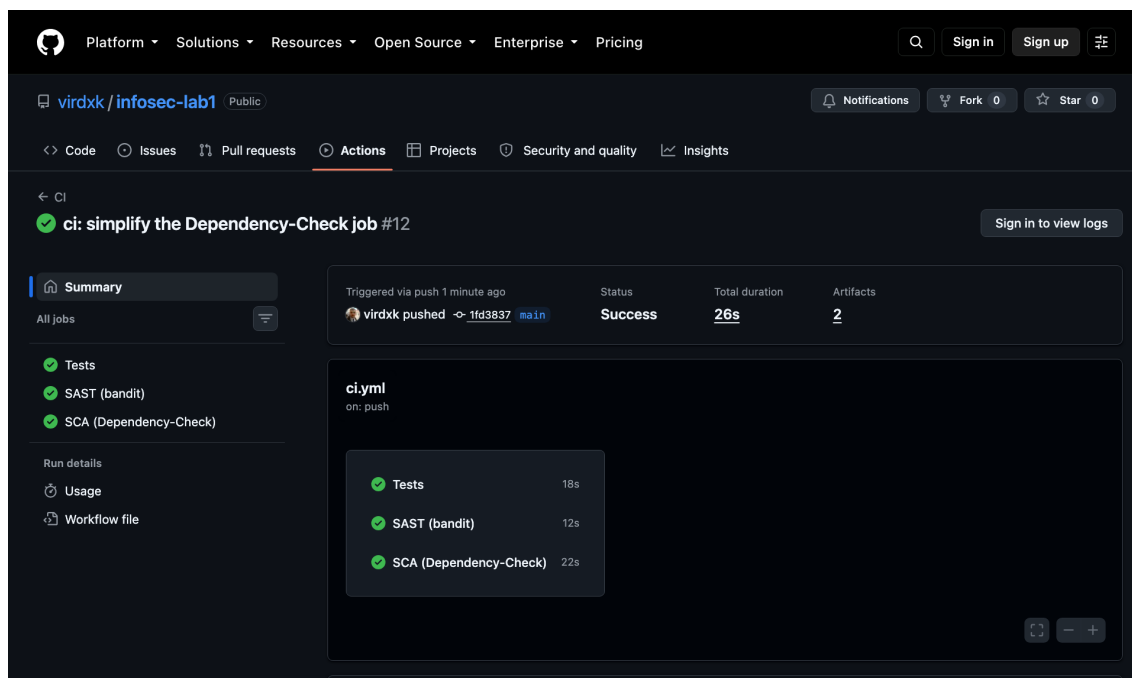


Рисунок 1 - Запуск CI #12: все три задачи завершились успешно, 2 артефакта

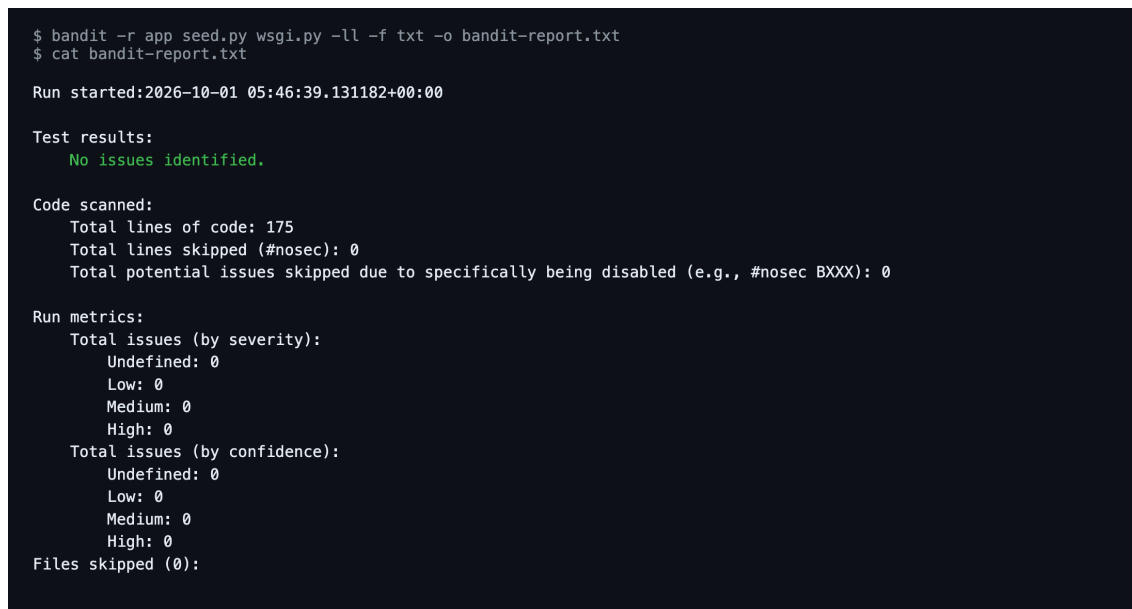


Рисунок 2 - Отчет Bandit из артефакта bandit-report



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

Project: infosec-lab1

Scan Information ([show all](#)):

- *dependency-check version:* 13.0.0
- *Report Generated On:* Thu, 1 Oct 2026 05:46:47 GMT
- *Dependencies Scanned:* 13 (13 unique)
- *Vulnerable Dependencies:* 0
- *Vulnerabilities Found:* 0
- *Vulnerabilities Suppressed:* 1 ([show](#))
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
------------	-------------------	---------	------------------	-----------	------------	----------------

Dependencies (vulnerable)

Suppressed Vulnerabilities

This report contains data retrieved from the [National Vulnerability Database](#).
This report may contain data retrieved from the [CISA Known Exploited Vulnerability Catalog](#).
This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).
This report may contain data retrieved from [RetireJS](#).
This report may contain data retrieved from the [Sonatype Guide OSS Index API](#).

Рисунок 3 - HTML-отчет OWASP Dependency-Check из артефакта dependency-check-report

Последний запуск пайплайна

Проверенный запуск, к которому относятся скриншоты и отчеты:

<https://github.com/virdxk/infosec-lab1/actions/runs/36821398841>

Все успешные запуски workflow:

<https://github.com/virdxk/infosec-lab1/actions/workflows/ci.yml?query=is%3Asuccess>